

Fundamentals of Computer Science II AE2

March 27, 2026

Introduction

For this assignment, I had to build a Python project that is connected to the Webtris API. The Webtris API is an API that gives us access to traffic data that is collected from sensors that have been placed around England. The sensor records things like average speed and how many cars pass every 15 minute intervals. My client file is made around three different classes: TrafficObservation, WebTRIS_API and SingleSite.

In this report, I am going to walk you through the decisions I made for designing the code, why I structured the code in the ways that I did and how I did the testing and decided what to test.

System Architecture

For this assignment, I split the webtris_client.py file into three different classes. They were called, `API()`, `SingleSite()`, and `TrafficObservation()`. I did it like because that's how our assignment brief asked us to do it but also it's good to separate what I want to do into separate classes instead of having it all in one class. It makes it easier to read and change the code later on.

The UML diagram I made shows how my three classes help each other for the program. I put `TrafficObservation()` at the top because all the other classes depend on it. This class stores all the data like `observation_date`, `time_period_ending`, `avg_speed` and `total_vehicles`. Each of the variables have type hints showing what they should return as.

API has an arrow pointing up to TrafficObservation. This is because API the class makes TrafficObservation objects inside the `fix_rows()` function.

Single Site has two different arrows coming from it. There is a solid line going up to the TrafficObservation class because SingleSite literally stores a list of the TrafficObservation objects inside its `traffic_stats` attribute. SingleSite also has a dependency arrow going to API because it only uses the class for a moment inside the `load_from_client()` function, but it doesn't actually store the API.

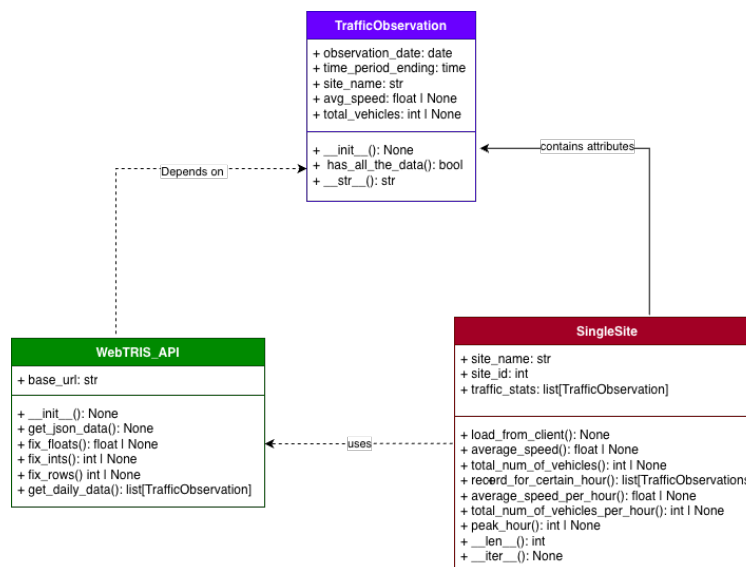


Figure 1: UML diagram

Design of the Classes

TrafficObservation

The first class I decided to start off with was TrafficObservation even though it was the last one we were said to do in the instructions. The reason I did this was because both of the other two classes depend on it. If I started with the other two then I would not have had anything to test them with.

The biggest thing that I thought of when writing this class was about missing data. The API could sometimes give missing data like empty strings, so I had to make that I account for things like that by storing values as None. I did not choose to store them as 0 because if I did that then the missing speed numbers would bring stuff like averages down.

```
avg_speed: float | None = None
total_vehicles: int | None = None
```

By doing None it would filter out those missing data and make sure not include those when trying to do our calculations. I made sure to make a function called has_all_data() that checks if average speed and total number of vehicles are not None that way I can check if the pytest will fail or not.

```
def has_all_the_data(self) -> bool:
    return self.avg_speed is not None and self.total_vehicles is not None
```

WebTRIS_API

One thing the brief mentioned was that the client could require more than one for it to work. I thought of doing this by splitting WebTRIS_API into two classes. One class would only handle the GET request and the other would handle the missing data and converting the data. I thought about when I was first writing the code but I realized that it would just add more complication to the project. I felt that the API class itself wasn't that much code so I could everything into one class. I made the decision though to separate many different things into different functions so that each function had one clear job and didn't include multiple functions in one.

For the WebTRIS_API class, the first thing I did was make a function which would call the API response. I made sure to make two separate methods. One would handle ONLY the API response and the errors that could come from it and the other would actually build the parameters. I had them as two different functions because if something were to go wrong in the request, I would know what the issue is easier.

The second thing that I made sure to do was thinking about how the API would send data. I realized that it would give me everything as a string so I had to create different functions that would turn the strings into floats or numbers. Instead of putting the fix_floats() and fix_ints() function inside of fix_row(), I had them as two individual functions. The main reason I did was for testing. I can test them easier when they are their own function, but also it makes it easier to read fix_rows() as well.

```
def fix_floats(self, value: str) -> float | None:
    if value == "":
        return None
    return float(value)
```

I made sure the fix_rows() function created new values where it would convert our attributes from strings into date, time, floats, or integers so we could turn them into TrafficObservation objects and store them in a list. Now from this list we can see all the information that we pulled from a certain day and from that we can calculate the information we need for our SingleSite class.

SingleSite

For the functions like `average_speed()`, `total_num_of_vehicles()`, `average_speed_per_hour()` and `total_num_of_vehicles_per_hour()`, I used same kind of pattern throughout them. I first filter out any observations where there is `None` using list comprehension. If the filtered list is empty I return `None` rather than trying to attempt a calculation. I made suer to do this because without doing the check where all speeds would be missing, there would be a divison error by zero and crash the whole program.

```
def average_speed(self) -> float | None:
    correct_speed_data = [mph_num for mph_num in self.traffic_stats if mph_num
                           .avg_speed is not None]

    if not correct_speed_data:
        return None

    total_speed = sum(mph_num.avg_speed for mph_num in correct_speed_data)
    return total_speed / len(correct_speed_data)
```

For `peak_hour()`, I had to think about the logic carefully. I set the busiest hour as 0 and most cars to whatever the value of `total_num_of_vehicles_per_hour(0)` returns. That number could also be `None` if the hour 0 has no data at all. I then loop through hours 1 to 23 and check if the curreent hour has more vehicles than the current busiest hour. The reason I check if `most_cars` is `None` inside the loop is to see if hour 0 has no data. If I dont do that check, then if `most_cars` started as `None` then the comparism would just give me a `TypeError` because you cannot compare an integer to a `None` in Python.

```
if most_cars is None or current_num_of_cars > most_cars:
    most_cars = current_num_of_cars
    busiest_hour = hour
```

In the loop, if we find that `current_num_of_cars` is more than `most_cars` then I set the most # of cars to the current num of cars. I also set the `busiest_hour` to the hour that the loop is currently on. Once the loop is completed, I return the hour that is the busiest car.

Error Handling

I approached error handling in two different ways in my code. The first place I did error handling was when I did the network errors. Inside `get_json_data()`, I look at three different exception types which were `Timeout`, `HTTPError` and `RequestException`. I also made sure to put these in a certain order. The reason I did this was because `Timeout` and `HTTPError` are both part of `RequestException`, which means if I had caught a `RequestException` first then it would just give me a general error. I made sure to do the most specific errors first and then the more general ones toward the end that way I catch all the different types of errors.

```
try:
    response = requests.get(...)
    response.raise_for_status()
    return response.json()
except Timeout:
    print("Timeout error. Try again later")
except HTTPError as e:
    print(f"HTTP error occurred: {e.response.status_code}")
except RequestException as e:
    print(f"Network error: {e}")
```

The second thing I did to account for error handling was think about what I would do about missing data. For missing data, I would handle that by storing any `None` for anything that the API would have as an empty

string. This was an alternative to using a value like 0 or raising an exception instead. Raising an exception everytime a data was missing would be useless and by storing 0 it would make the alter the calculations. If I wanted to find the average of something in SingleSite, by setting the missing value to 0, I would not get the correct average and I would be messing up the data. By setting every missing data value to None, that would mean I could just skip over the missing data values and still end up calculating my calculations.

User Usage

I wrote the code for main.py in a way so that the user has to change as little code as possible to get actual results. All the user needs to do is change three things, which is the `site_id`, `site_name`, and `date`. Everything else will run by itself from there once they have put in these values. I let the user know as well that to find a site id and site name that is valid they can go to a certain link in the browser to see all the available site ids and site names.

```
https://webtris.nationalhighways.co.uk/api/v1.0/sites
```

This shows them a full list of every single sensor with its ID and its name. Once they choose one, they only need to change the code shown below.

```
site = SingleSite(site_id=461, site_name="M25/4432A", traffic_stats=[])
site.load_from_client(client, date(2026, 1, 3))
```

From these two lines of code, we are able to get all the 96 observations for that day. I also made sure to make the output in main.py visually easy to read by using separator lines by using "=" to separate the sections. This breaks all the different outputs I have into clear sections so that the user can easily find the information they are looking for. An example of that is shown below.

Output in Terminal (Jan 3rd, 2026)

```
=====
Total number of records: 96
Average speed for the day: 64.4375 mph
Total number of vehicles for the day: 58059
Peak Hour: 11:00 to 12:00
Average speed during peak hour: 64.75 mph
Total number of vehicles during peak hour: 4843
=====

All the records for hour 8:
M25/4432A 2026-01-03 08:14:00, Total Vehicles = 519, Speed = 67.0
M25/4432A 2026-01-03 08:29:00, Total Vehicles = 570, Speed = 67.0
M25/4432A 2026-01-03 08:44:00, Total Vehicles = 614, Speed = 67.0
M25/4432A 2026-01-03 08:59:00, Total Vehicles = 625, Speed = 66.0
=====

All the records for a certain date:
M25/4432A 2026-01-03 00:14:00, Total Vehicles = 147, Speed = 65.0
M25/4432A 2026-01-03 00:29:00, Total Vehicles = 162, Speed = 65.0
...
M25/4432A 2026-01-03 23:59:00, Total Vehicles = 193, Speed = 65.0
```